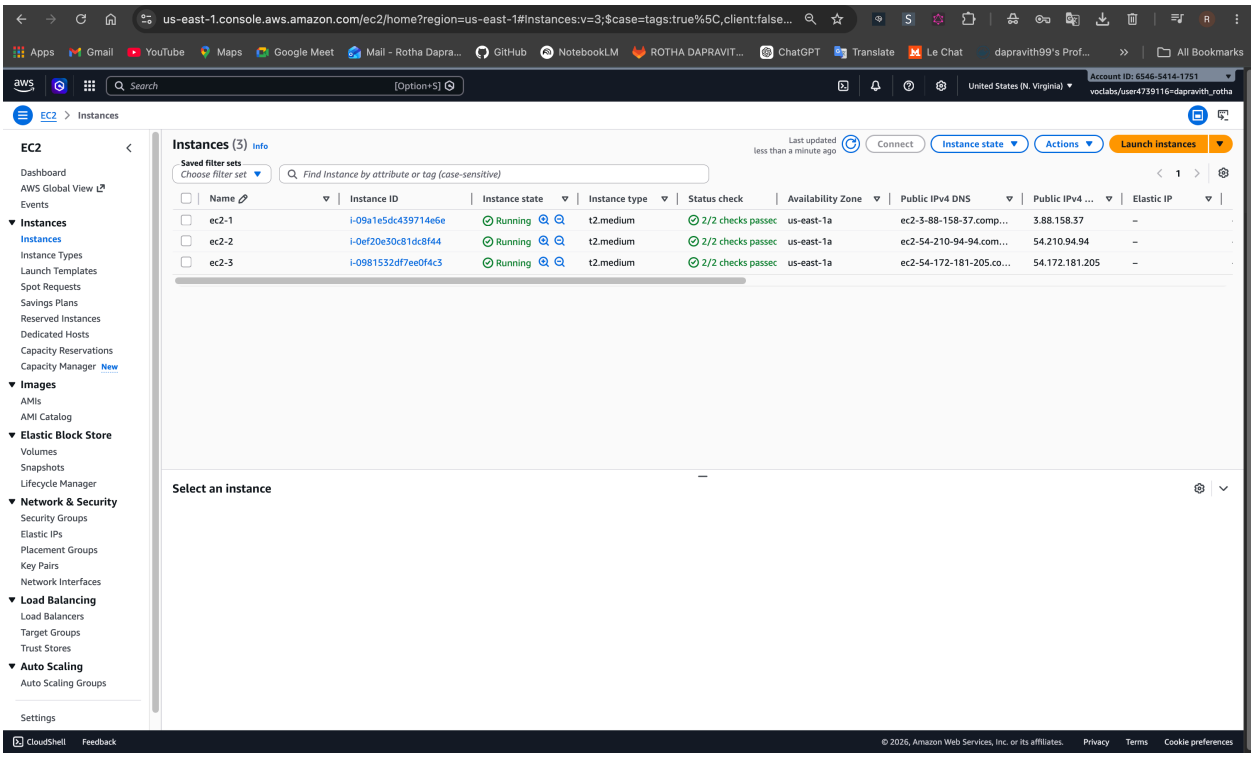
Project microservices-docker-k8s

GitHub Repository: https://github.com/Dapravith/microservices-docker-k8s

1. 3 EC-2 instances



2. Docker image / docker container of the 3 EC2 instances

```
IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
dapravith99/api-gateway:1.0.0        a695fce6599f       549MB         148MB
dapravith99/login-service:1.0.0      c06d39ce98de       544MB         146MB
dapravith99/student-service:1.0.0    f4437a3fda79       536MB         142MB
dapravith99/teacher-service:1.0.0    a469ea31f237       536MB         142MB
```

3. 3 Services and deployment of the 3 EC2 instances

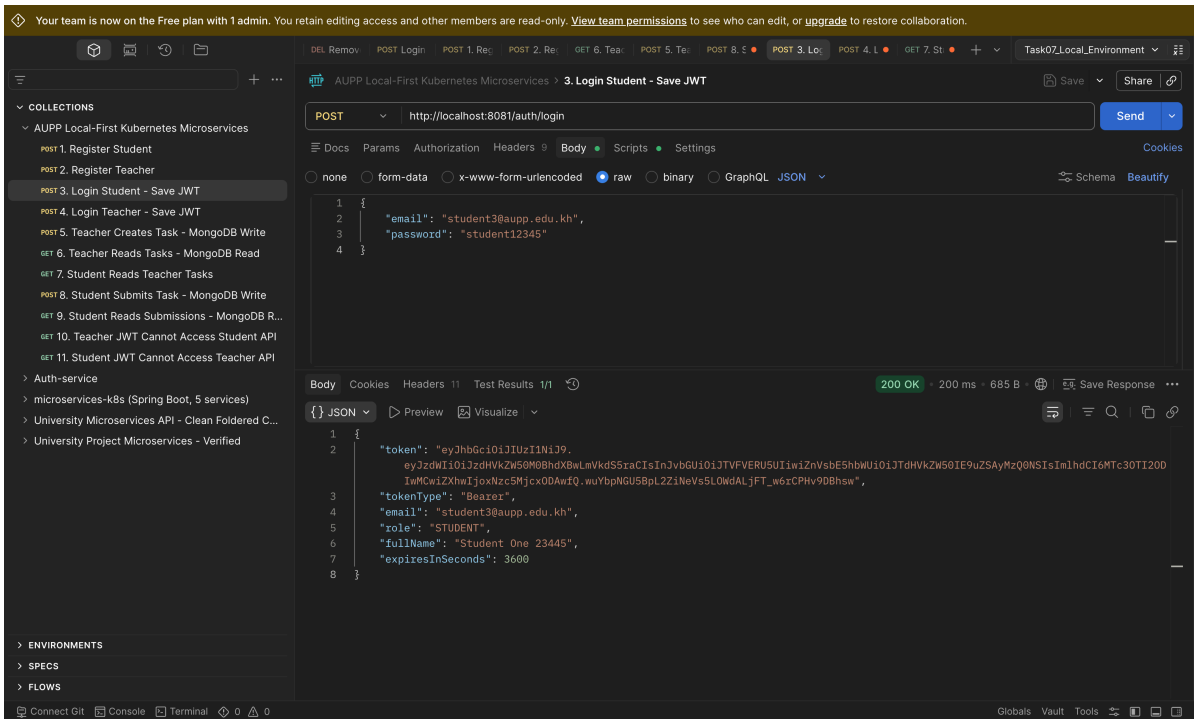
```
NAME                   STATUS   ROLES           AGE   VERSION   LABELS
aupp-local-control-plane   Ready    control-plane   47h   v1.34.0   beta.kubernetes.io/arch=arm64,beta.kubernetes.io/os=linux,kubernetes.io/arch=arm64,kubernetes.io/hostname=aupp-local-control-plane,kubernetes.io/os=linux,node-role.kubernetes.io/control-plane=
,node.kubernetes.io/exclude-from-external-load-balancers=,role=admin
aupp-local-worker          Ready    <none>          47h   v1.34.0   beta.kubernetes.io/arch=arm64,beta.kubernetes.io/os=linux,kubernetes.io/arch=arm64,kubernetes.io/hostname=aupp-local-worker,kubernetes.io/os=linux,role=student
aupp-local-worker2         Ready    <none>          47h   v1.34.0   beta.kubernetes.io/arch=arm64,beta.kubernetes.io/os=linux,kubernetes.io/arch=arm64,kubernetes.io/hostname=aupp-local-worker2,kubernetes.io/os=linux,role=teacher
```

4. pods screenshot inside 3 ec2 instance

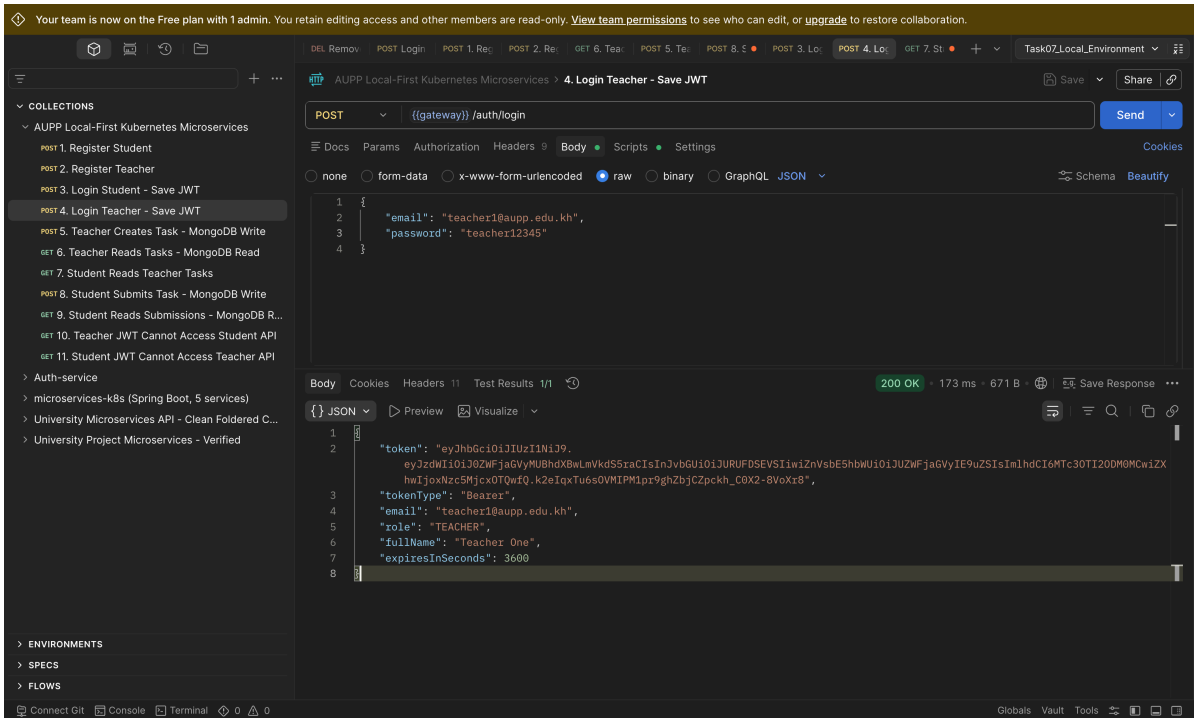
```
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS        IMAGES                                  SELECTOR
deployment.apps/api-gateway         1/1     1            1           47h   api-gateway       dapravith99/api-gateway:1.0.0           app=api-gateway
deployment.apps/login-service       1/1     1            1           47h   login-service     dapravith99/login-service:1.0.0         app=login-service
deployment.apps/student-service     1/1     1            1           47h   student-service   dapravith99/student-service:1.0.0       app=student-service
deployment.apps/teacher-service     1/1     1            1           47h   teacher-service   dapravith99/teacher-service:1.0.0       app=teacher-service
```

```
NAME                            READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS        IMAGES                                  SELECTOR
deployment.apps/api-gateway     1/1     1            1           47h   api-gateway       dapravith99/api-gateway:1.0.0           app=api-gateway
deployment.apps/login-service   1/1     1            1           47h   login-service     dapravith99/login-service:1.0.0         app=login-service
deployment.apps/student-service 1/1     1            1           47h   student-service   dapravith99/student-service:1.0.0       app=student-service
deployment.apps/teacher-service 1/1     1            1           47h   teacher-service   dapravith99/teacher-service:1.0.0       app=teacher-service

NAME                     READY   AGE   CONTAINERS   IMAGES
statefulset.apps/mongodb 1/1     47h   mongodb      mongo:7

NAME                     TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE   SELECTOR
service/api-gateway      NodePort    10.96.186.188   <none>        8080:30080/TCP   47h   app=api-gateway
service/login-service    ClusterIP   10.96.95.109    <none>        8081/TCP         47h   app=login-service
service/mongodb          ClusterIP   10.96.161.158   <none>        27017/TCP        47h   app=mongodb
service/student-service  ClusterIP   10.96.158.103   <none>        8082/TCP         47h   app=student-service
service/teacher-service  ClusterIP   10.96.195.163   <none>        8083/TCP         47h   app=teacher-service

NAME                                         STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE   VOLUMEMODE
persistentvolumeclaim/mongodb-data-mongodb-0 Bound    pvc-cd4bbf9a-e2eb-41ce-9148-96f0f1c699d8   1Gi        RWO            standard       <unset>                 47h   Filesystem

NAME                           READY   STATUS        RESTARTS      AGE    IP            NODE                     NOMINATED NODE   READINESS GATES
api-gateway-59fb9b5794-bsz9f   1/1     Running       0             33s    10.244.0.10   aupp-local-control-plane   <none>           <none>
api-gateway-67bb6f9df6-vthd6   1/1     Terminating   0             155m   10.244.0.9    aupp-local-control-plane   <none>           <none>
login-service-5cb47d7849-z7fhw 1/1     Terminating   0             155m   10.244.0.8    aupp-local-control-plane   <none>           <none>
login-service-85cc5f99f9-bdkdc 1/1     Running       0             33s    10.244.0.11   aupp-local-control-plane   <none>           <none>
mongodb-0                      1/1     Running       1 (26h ago)   47h    10.244.0.6    aupp-local-control-plane   <none>           <none>
student-service-5748477b5f-g252v 1/1   Running       0             33s    10.244.2.4    aupp-local-worker          <none>           <none>
student-service-78b54679c-k759c 1/1    Terminating   0             155m   10.244.2.3    aupp-local-worker          <none>           <none>
teacher-service-76988b9649-mdfvc 1/1   Terminating   0             155m   10.244.1.3    aupp-local-worker2         <none>           <none>
teacher-service-dbd498695-6d8sc 1/1    Running       0             33s    10.244.1.4    aupp-local-worker2         <none>           <none>

Gateway (NodePort, all endpoints): http://localhost:30080
rothadapravith@ROTHAs-MacBook-Pro local %
```

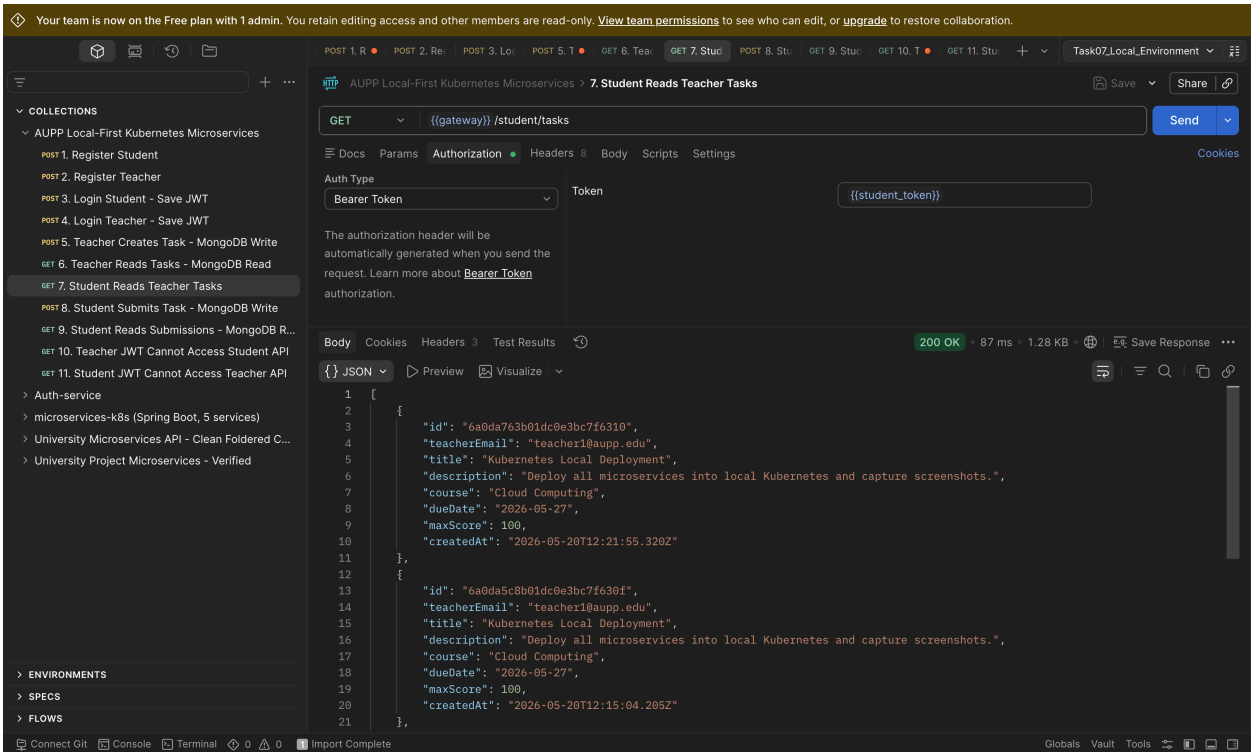
5. Login using Postman and return JWT token
    a. Student Login



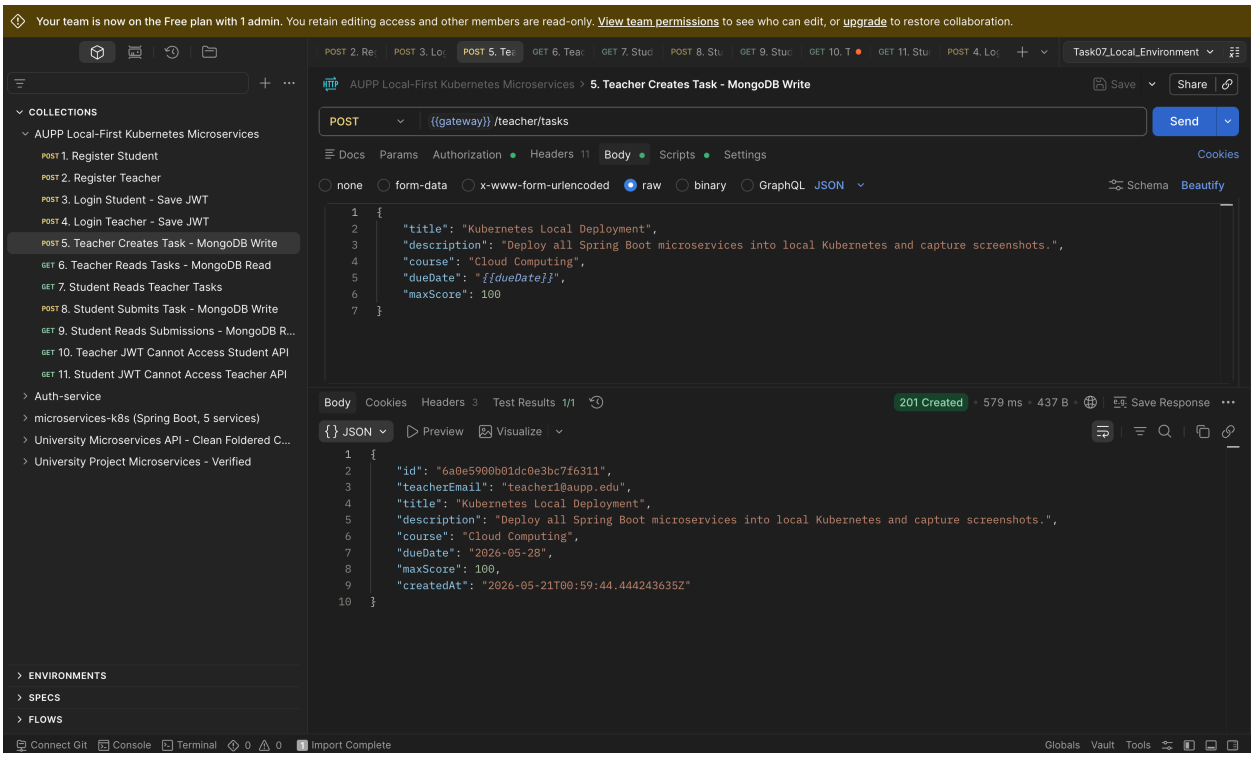
b. Teacher Login



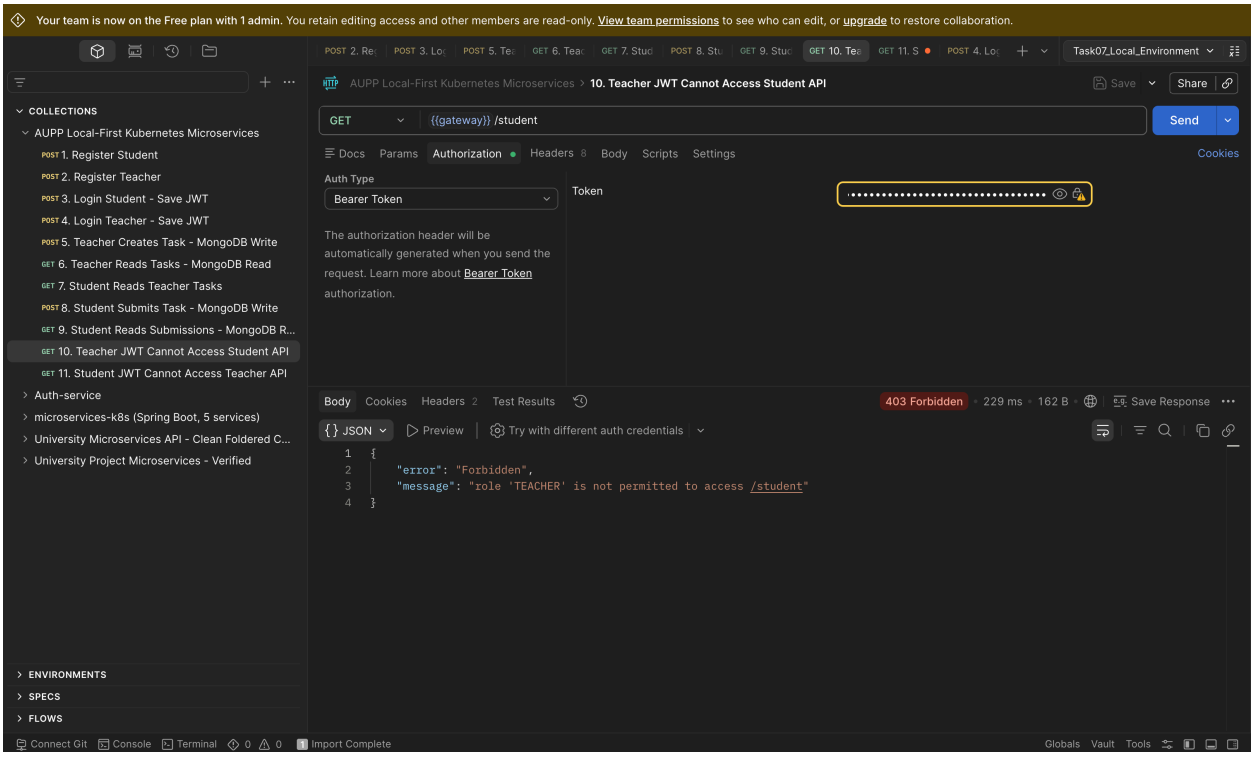
6. /student with student JWT - it will access student API which should have some
   database activity



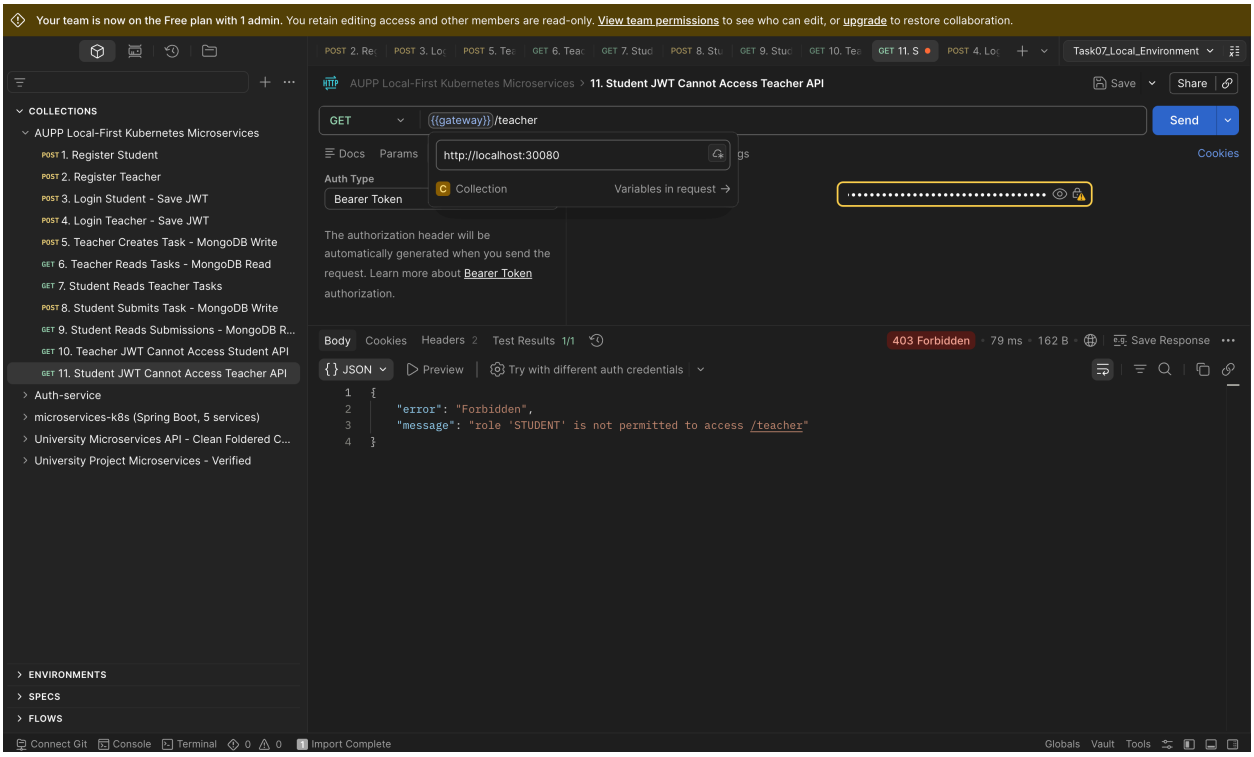
7. /teacher with teacher JWT - it will access student API which should have some
   database activity



1. /student with teacher JWT – Will not work



1. /teacher with student JWT – Will not work



Workaround for your current EC2

Run on **EC2-1 only**:

```
sudo kubeadm init \
--pod-network-cidr=10.244.0.0/16 \
--ignore-preflight-errors=Mem
```

If it succeeds, configure `kubectl`:

```
mkdir -p $HOME/.kube
sudo cp -f /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Install Flannel:

```
kubectl apply -f https://github.com/flannel-io/flannel/releas
es/latest/download/kube-flannel.yml
```

Check node:

```
kubectl get nodes -o wide
kubectl get pods -A
```

Run in local

Step 1 — Deploy

./scripts/local/deploy.sh

This single script does everything:

- create-kind-cluster.sh — creates the 3-node kind cluster + labels nodes (admin/student/teacher)
- build-images.sh — builds all 4 service images
- kind load — loads images into the cluster
- kubectl apply -k deploy/k8s + rollout restart + waits for rollout

Step 2 — Verify

./scripts/test-postman-flow.sh

Defaults to http://localhost:30080, no env var needed.

Notes:

- You don't run create-kind-cluster.sh or build-images.sh separately — deploy.sh calls them for you.
- push-images.sh is optional — only if you want to push to Docker Hub.
- Cleanup when done: ./scripts/local/delete-kind-cluster.sh

So: local = 2 commands (deploy.sh → test-postman-flow.sh), while EC2 = staged across 3 machines because the cluster has to be bootstrapped manually.

Run in ec2-instance

Step 1 — Bootstrap the cluster

On ec2-1 (server / admin node):
cd microservices-docker-k8s
NODE_NAME=ec2-1 ./scripts/ec2/bootstrap-k3s-server.sh
→ This prints the K3S token and server URL — copy them.

On ec2-2 (agent / student node):
cd microservices-docker-k8s
K3S_URL=https://<ec2-1-private-ip>:6443 K3S_TOKEN=<token>
NODE_NAME=ec2-2 ./scripts/ec2/bootstrap-k3s-agent.sh

On ec2-3 (agent / teacher node):
cd microservices-docker-k8s
K3S_URL=https://<ec2-1-private-ip>:6443 K3S_TOKEN=<token>
NODE_NAME=ec2-3 ./scripts/ec2/bootstrap-k3s-agent.sh

Step 2 — Build images on each node

ec2-1

NODE_ROLE=admin ./scripts/ec2/build-node-images.sh

ec2-2

NODE_ROLE=student ./scripts/ec2/build-node-images.sh

ec2-3

NODE_ROLE=teacher ./scripts/ec2/build-node-images.sh

Step 3 — Deploy (run only on ec2-1)

EC2_1_NODE=ec2-1 EC2_2_NODE=ec2-2 EC2_3_NODE=ec2-3 ./scripts/ec2/deploy.sh

Step 4 — Verify (from anywhere)

GATEWAY=http://<ec2-1-public-ip>:30080 ./scripts/test-postman-flow.sh

Notes:

- label-nodes.sh does not need to be run separately — deploy.sh calls it for you.
- Order matters: the agents (Step 1) must join before you build/deploy, and all images (Step 2) must exist before deploy.sh, otherwise pods fail with ImagePullBackOff (imagePullPolicy: IfNotPresent + no registry).